REDBUS · DEPLOYMENT GUIDE

redBus Website Deployment Jenkins CI/CD on AWS EC2

A complete, step-by-step guide to deploying the redBus bus-booking website: from GitHub to Docker Hub to a live container on AWS EC2, automated with a Jenkins Freestyle pipeline.

redBus | Jenkins | Docker | GitHub | AWS EC2



PROJECT
CICD-Project-redbus

VERSION
1.0

DATE
19 October 2026

PREPARED BY
Mayur Chaudhari

Contents

DOCUMENT	PROJECT
Jenkins CI/CD on AWS EC2: Implementation Guide	CICD-Project-redbus
VERSION	DATE
1.0	19 October 2026

01 Overview 3
02 Provision the AWS EC2 Instance 4
03 Install and Configure Jenkins 8
04 Install Git and Docker 10
05 Prepare the Project and Push to GitHub 12
06 Create a Docker Hub Access Token 21
07 Store Credentials in Jenkins 22
08 Create and Configure the Freestyle Job 23
09 Run and Verify the Pipeline 28
10 Troubleshooting 30
11 Security Recommendations 30
· Appendix A: Command Reference 31
· Appendix B: Port Reference 32

01 Overview

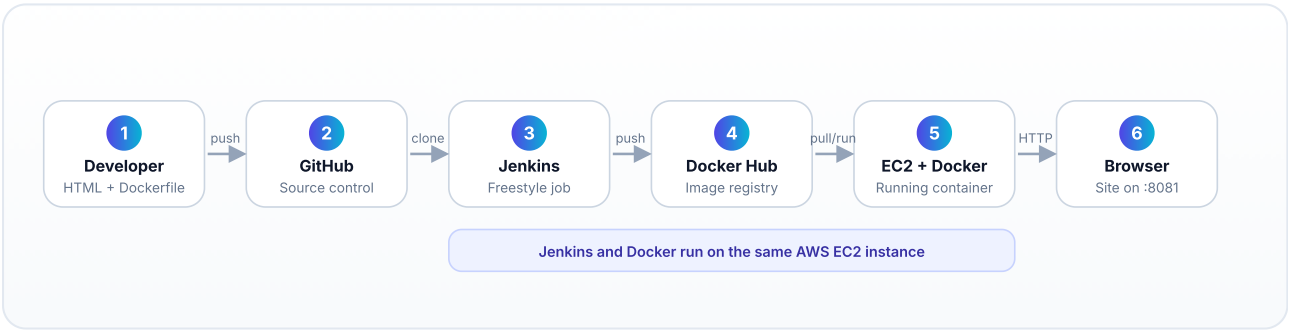
This document describes how the **CICD-Project-redbus** website was built, containerised and deployed using a Jenkins Freestyle job running on an AWS EC2 instance. Every action is documented with the exact commands used and annotated screenshots from the live environment, so the setup can be reproduced end to end.

CLOUD	CI SERVER	RUNTIME	LIVE PORT
AWS EC2	Jenkins 2.580.1	Docker	8081

Objectives

- Provision and secure an AWS EC2 instance for Jenkins and Docker.
- Install Jenkins, Java 21, Git and Docker on Amazon Linux 2023.
- Store the website source in GitHub and package it as a Docker image.
- Authenticate to Docker Hub with a scoped access token held in Jenkins credentials.
- Automate build, push and deploy with a Freestyle job, then verify the live site.

Solution architecture



Environment summary

COMPONENT	DETAILS
Cloud provider	Amazon Web Services, region **eu-north-1** (Stockholm)
Instance	Name **MY-Project**, type **c7i-flex.large** (2 vCPU, 4 GiB RAM)
Operating system	Amazon Linux 2023 (kernel 6.18 AMI)
Storage	25 GiB gp3 root volume
Java	Amazon Corretto 21
CI server	Jenkins 2.580.1, reachable on port **8080**
Source control	GitHub public repository `CICD-Project-redbus`
Registry	Docker Hub, authenticated with a personal access token
Application	Static website served from a container on port **8081**

Prerequisites

- An AWS account with permission to launch EC2 instances and edit security groups.
- A GitHub account and a Docker Hub account.
- A web browser (AWS Console access uses EC2 Instance Connect, so no local SSH client is required).
- Basic familiarity with the Linux command line.

02 Provision the AWS EC2 Instance

The instance hosts both Jenkins and the application container, so it needs enough memory and disk to build Docker images.

Name the instance and select the AMI

1. Open the AWS Console, search for **EC2** and choose **Instances → Launch instances**.
2. Set **Name** to `MY-Project`.
3. Under **Application and OS Images**, choose **Amazon Linux** and keep the Amazon Linux 2023 AMI (free tier eligible).

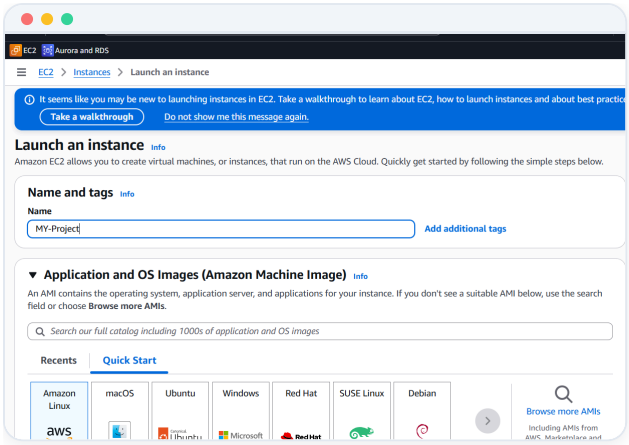


Figure 1. Launch an instance: name and tags

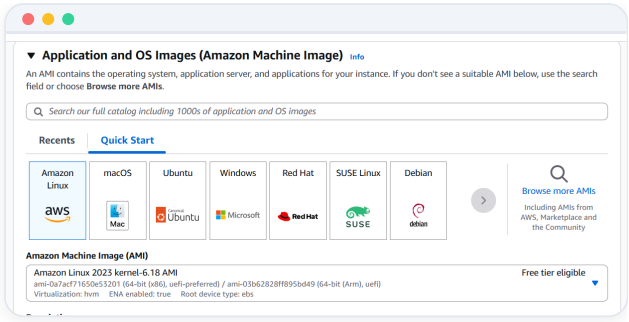


Figure 2. Amazon Linux 2023 AMI selected

Choose instance type and key pair

The `c7i-flex.large` type (2 vCPU, 4 GiB memory) gives Jenkins and Docker enough headroom. No key pair was created because the instance is accessed through **EC2 Instance Connect** in the browser.

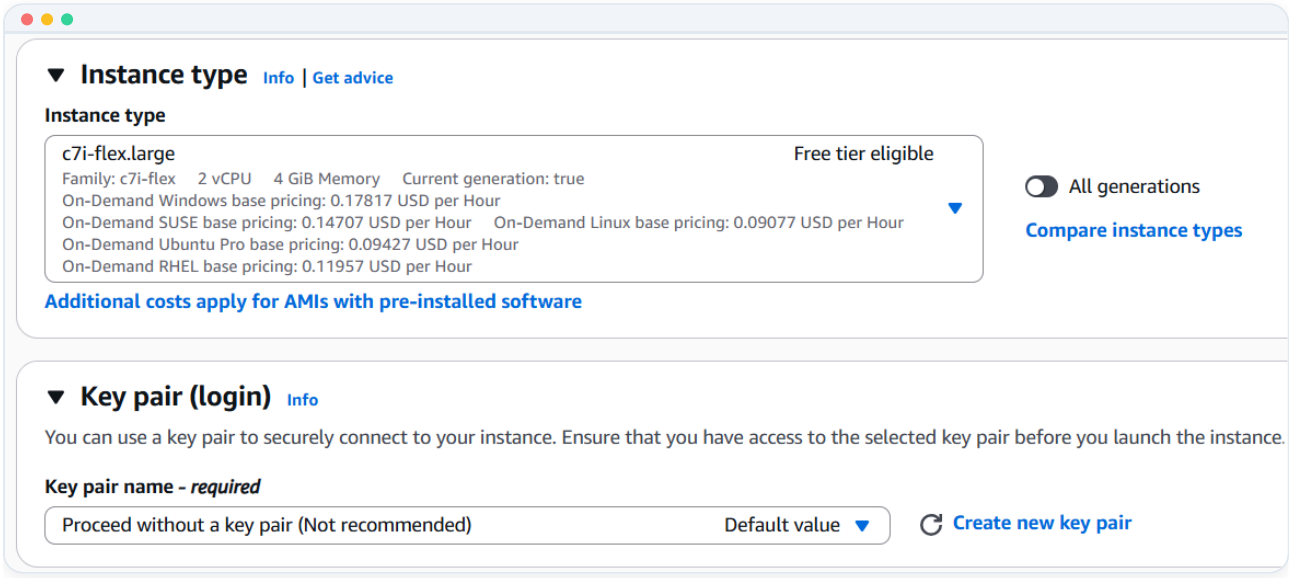


Figure 3. Instance type and key pair settings

Network settings and storage

A new security group (`launch-wizard-1`) was created allowing SSH (22), HTTP (80) and HTTPS (443). Root storage was set to 25 GiB gp3.

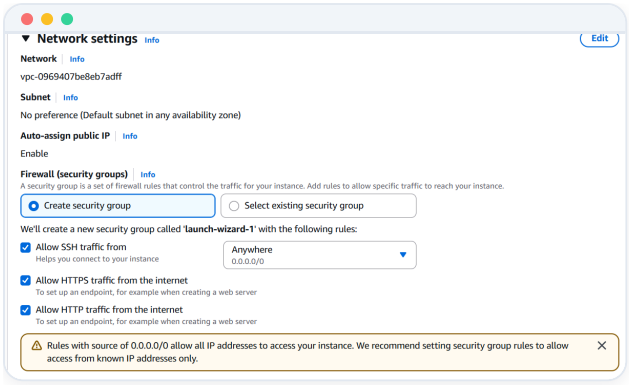


Figure 4. Network settings and firewall rules

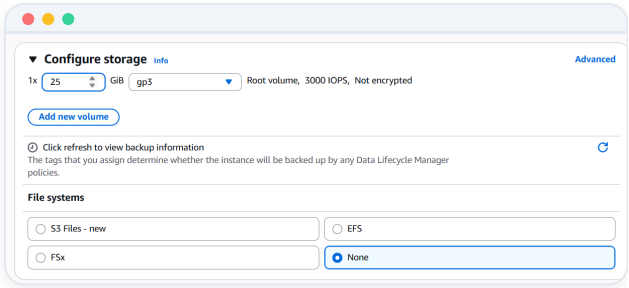


Figure 5. Configure storage: 25 GiB gp3

Review and launch

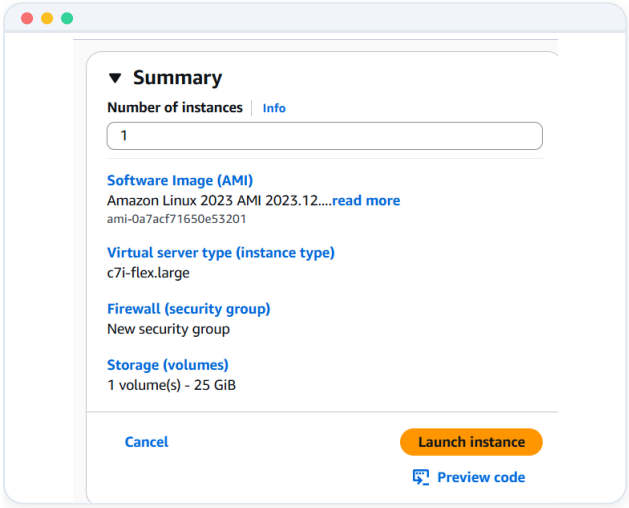


Figure 6. Launch summary

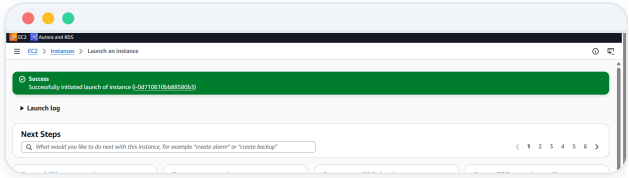


Figure 7. Instance launch initiated successfully

Verify the instance state

Wait until the instance shows **Running** and the status checks pass. Note the **Public IPv4 address**; it is used for every browser URL in this guide.

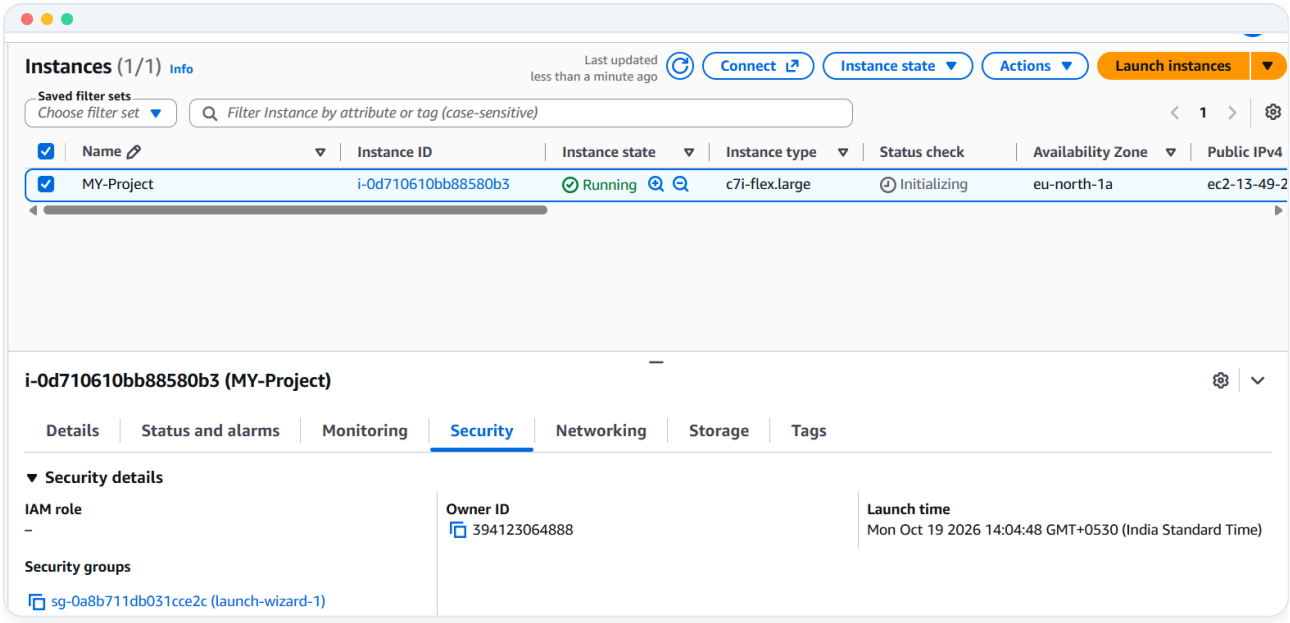


Figure 8. Instance running, with the Security tab open

Open ports 8080 and 8081

From the Security tab, open the security group and choose **Edit inbound rules**. Add two **Custom TCP** rules and select **Save rules**.

TCP 22	TCP 8080	TCP 8081
SSH	**Jenkins UI**	**Website**
Remote shell access	CI server web interface	Application container output

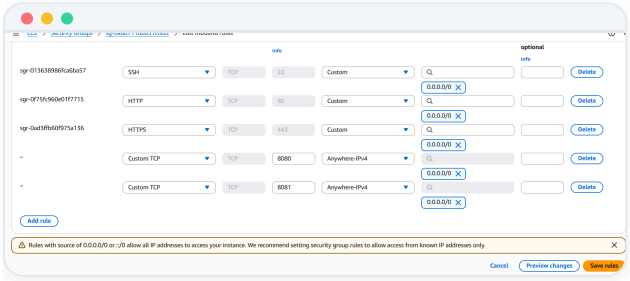


Figure 9. Edit inbound rules: 8080 and 8081 added

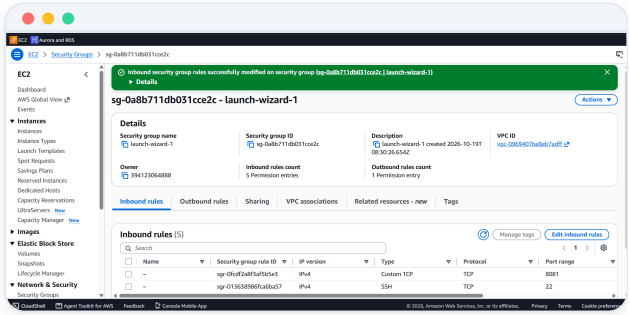


Figure 10. Security group rules saved

> **Restrict public access**
> This lab allows `0.0.0.0/0` (anywhere). For anything beyond a lab, limit ports 22 and 8080 to your own IP address.

03 Install and Configure Jenkins

Connect to the instance

Select the instance and choose **Connect**. On the **EC2 Instance Connect** tab keep **Public subnet access** and click **Connect** to open a browser-based terminal.

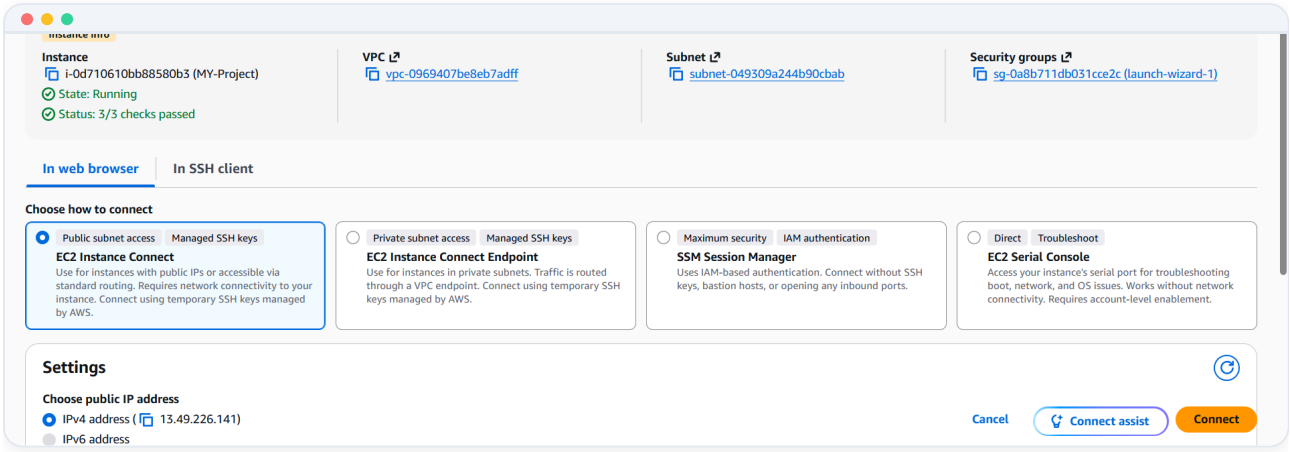


Figure 11. EC2 Instance Connect: connect via web browser

Switch to the root user so that the following commands run without repeating `sudo`.

```
$ sudo su
```

> **About the prompt**
> The prompt changes from `ec2-user` to `root`. Commands in this guide that start with `sudo` work for either user.

Update the system and install Java

```
$ yum update -y
$ sudo dnf install -y java-21-amazon-corretto
```

COMMAND	PURPOSE
`yum update -y`	Updates all installed packages. `-y` answers "yes" to every prompt. "Nothing to do. Complete!" means the system is current.
`sudo dnf install -y java-21-amazon-corretto`	Installs Amazon Corretto 21, the Java runtime Jenkins requires.

Add the Jenkins repository and install

```
$ sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
$ sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
$ sudo dnf install -y jenkins
```

COMMAND	PURPOSE
`wget -O /etc/yum.repos.d/jenkins.repo <url>`	Downloads the Jenkins repository definition so the package manager knows where to find Jenkins.
`rpm --import <key>`	Imports the repository signing key so package signatures can be verified.
`dnf install -y jenkins`	Installs the Jenkins server.

Signing key errors
If the install reports a GPG key error, import the current key `jenkins.io-2026.key` from the same location and run the install again.

Start Jenkins

```
$ sudo systemctl start jenkins
$ sudo systemctl enable jenkins
$ sudo systemctl status jenkins
```

COMMAND	PURPOSE
`systemctl start jenkins`	Starts the Jenkins service now.
`systemctl enable jenkins`	Starts Jenkins automatically after every reboot.
`systemctl status jenkins`	Shows the service state. Look for `active (running)`.

Unlock Jenkins

1. Browse to `http://<EC2_PUBLIC_IP>:8080`.
2. Read the initial administrator password on the server (command below) and paste it into the Unlock screen.
3. Choose **Install suggested plugins** and wait for the installation to finish.
4. Create the admin account, confirm the Jenkins URL and select **Start using Jenkins**.

```
$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

`cat` prints the file that holds the one-time administrator password generated during installation.

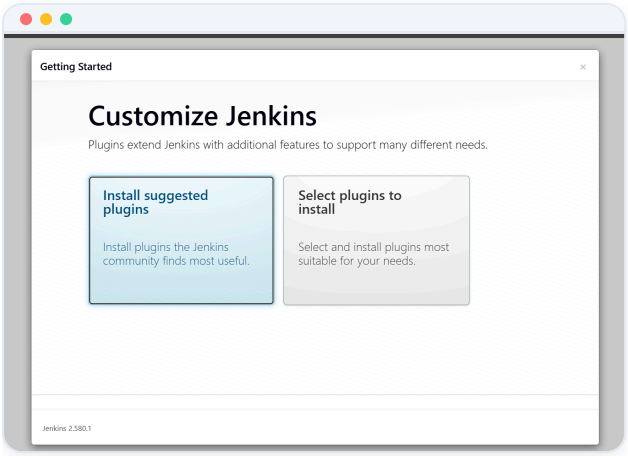


Figure 12. Customize Jenkins: install suggested plugins

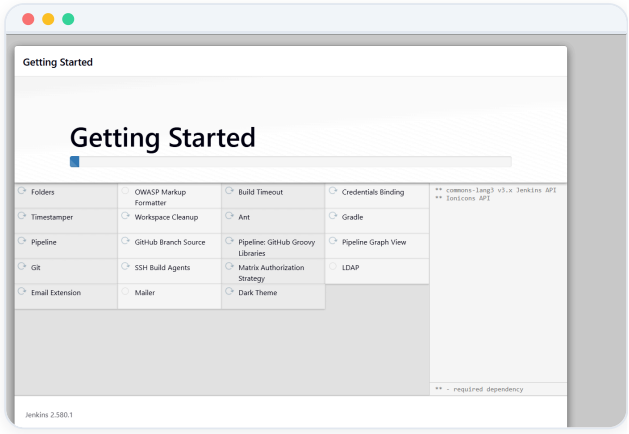


Figure 13. Suggested plugins installing

Checkpoint

The Jenkins dashboard is reachable and the suggested plugins (including Git and Credentials Binding) are installed.

04 Install Git and Docker

Install Git

```
$ sudo dnf install -y git
$ git --version
```

Jenkins uses Git to fetch the website source from GitHub. `git --version` confirms the installation.

Install and start Docker

```
$ yum install docker -y
$ docker -v
$ systemctl start docker
$ systemctl enable docker
$ systemctl status docker
```

COMMAND	PURPOSE
`yum install docker -y`	Installs the Docker engine.
`docker -v`	Prints the installed Docker version.
`systemctl start docker`	Starts the Docker service.
`systemctl enable docker`	Enables Docker at boot.
`systemctl status docker`	Confirms the service is running.

Grant Docker access to Jenkins

Jenkins runs as its own `jenkins` Linux user, which must belong to the `docker` group to run container commands.

```
$ sudo usermod -aG docker jenkins
$ sudo usermod -aG docker ec2-user
$ docker ps
$ sudo systemctl restart jenkins
$ docker ps
```

COMMAND	PURPOSE
`usermod -aG docker jenkins`	Appends (`-a`) the jenkins user to the docker group (`-G`).
`usermod -aG docker ec2-user`	Does the same for ec2-user.
`docker ps`	Lists running containers. An empty list without errors confirms permissions.
`systemctl restart jenkins`	Restarts Jenkins so the new group membership applies.

Security note
Membership in the `docker` group is effectively root-level access to the host. This shortcut is acceptable for a learning lab; in production use dedicated build agents.

05 Prepare the Project and Push to GitHub

Create the project files

```
$ mkdir cicd-project
$ cd cicd-project
$ vi index.html
$ vi Dockerfile
$ cat Dockerfile
```

COMMAND	PURPOSE
`mkdir cicd-project`	Creates the project directory.
`cd cicd-project`	Moves into it.
`vi index.html`	Opens the editor for the website page. Press `i` to insert, `Esc` then `:wq` to save.
`vi Dockerfile`	Creates the Dockerfile that describes the image.
`cat Dockerfile`	Prints the file to confirm its content.

Typical static-site Dockerfile
A static website usually needs only three instructions: `FROM nginx:alpine`, `COPY index.html /usr/share/nginx/html/index.html` and `EXPOSE 80`.

Build the image locally

```
$ docker build -t jiohostar .
```

`-t jiohostar` names (tags) the image and the trailing `.` tells Docker to use the Dockerfile in the current directory. A successful build proves the Dockerfile works before Jenkins automates it.

Create the GitHub repository

In GitHub choose **New repository**, name it `CICD-Project-redbus`, set visibility to **Public** (so Jenkins can clone without credentials) and click **Create repository**.

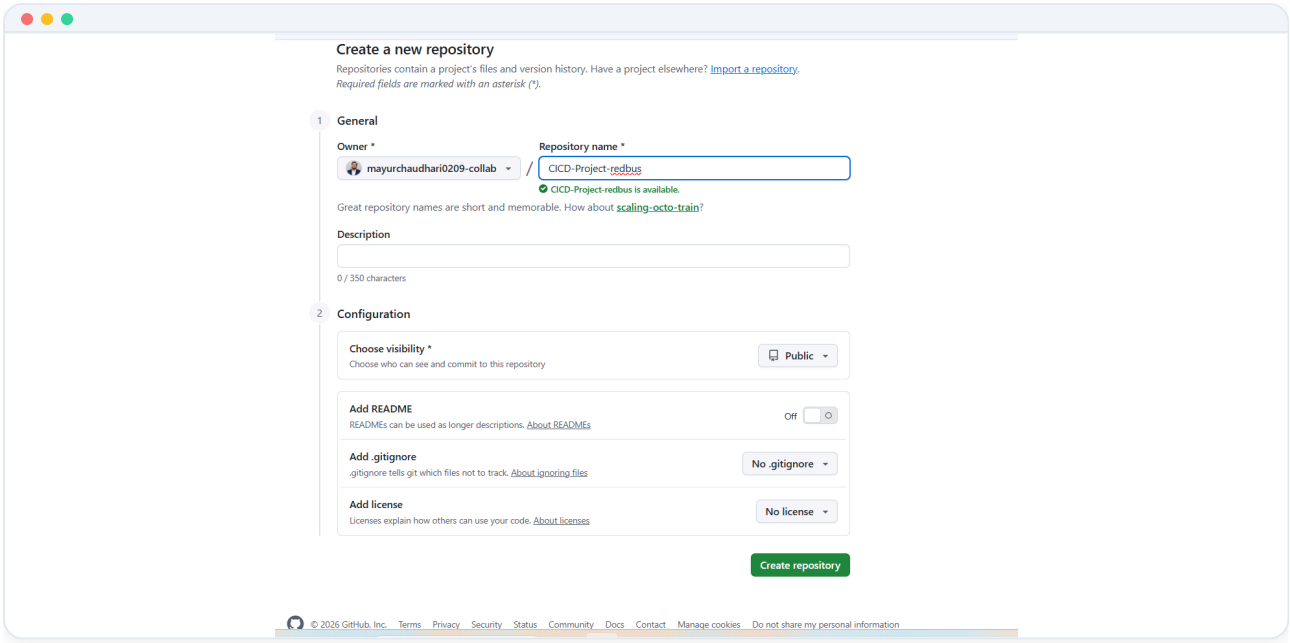


Figure 14. GitHub: create a new repository

Prepare the project files

On your computer (or the EC2 terminal), create a folder with two files inside. Both files are committed to GitHub in the next step.

```
cicd-project/
├── index.html
└── Dockerfile
```

Create index.html. Paste the following into `index.html`. This is the redBus bus-booking landing page (HTML, CSS and a little JavaScript) that the pipeline builds into a Docker image and deploys.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>RedBus - Bus Booking</title>

<style>
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
}

body {
    background: #f5f5f5;
    color: #333;
}

/* Navigation */
nav {
```

```
    background: white;
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 18px 8%;
    box-shadow: 0 3px 15px #0001;
    position: sticky;
    top: 0;
    z-index: 10;
}

.logo {
    font-size: 30px;
    font-weight: bold;
    color: #d92332;
    letter-spacing: -1px;
}

nav a {
    text-decoration: none;
    color: #444;
    margin-left: 25px;
    font-weight: bold;
}

nav a:hover {
    color: #d92332;
}

/* Hero */
.hero {
    min-height: 480px;
    padding: 65px 7% 85px;
    text-align: center;
    color: white;
    background:
        linear-gradient(120deg, #a80013ee, #ed2639dd),
        #d92332;
    position: relative;
    overflow: hidden;
}

.hero h1 {
    font-size: clamp(30px, 5vw, 48px);
    margin-bottom: 15px;
    animation: fadeIn 1s ease;
}

.hero p {
    font-size: 18px;
    margin-bottom: 35px;
}

.booking {
    max-width: 1000px;
    margin: auto;
    background: white;
    color: #333;
    padding: 22px;
    border-radius: 16px;
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 15px;
    box-shadow: 0 15px 40px #57000b55;
```

```
}

.field {
    text-align: left;
}

.field label {
    display: block;
    margin-bottom: 9px;
    font-size: 13px;
    color: #666;
    font-weight: bold;
}

.field input {
    width: 100%;
    padding: 13px 10px;
    border: 1px solid #ddd;
    border-radius: 8px;
    outline: none;
    font-size: 15px;
}

.field input:focus {
    border-color: #d92332;
    box-shadow: 0 0 5px #d9233244;
}

.search-btn {
    grid-column: 1 / -1;
    border: none;
    background: #d92332;
    color: white;
    padding: 16px;
    font-size: 17px;
    border-radius: 9px;
    cursor: pointer;
    font-weight: bold;
    transition: .3s;
}

.search-btn:hover {
    background: #a80013;
    transform: translateY(-2px);
}

/* Moving bus */
.road {
    position: absolute;
    bottom: 0;
    left: 0;
    width: 100%;
    height: 55px;
    background: #282828;
}

.road::after {
    content: "";
    position: absolute;
    top: 26px;
    width: 100%;
    border-top: 3px dashed white;
}
```

```
.bus {
    position: absolute;
    bottom: 45px;
    left: -120px;
    font-size: 65px;
    animation: drive 12s linear infinite;
}

@keyframes drive {
    from { left: -120px; }
    to { left: 110%; }
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(20px); }
    to { opacity: 1; transform: translateY(0); }
}

/* Offers */
.container {
    padding: 45px 8%;
}

.container h2 {
    font-size: 28px;
    margin-bottom: 25px;
}

.offers {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}

.offer {
    padding: 25px;
    border-radius: 15px;
    color: white;
    background: linear-gradient(135deg, #e52d3e, #a80013);
    box-shadow: 0 7px 20px #0001;
    transition: .3s;
}

.offer:nth-child(2) {
    background: linear-gradient(135deg, #ff7a18, #e52d3e);
}

.offer:nth-child(3) {
    background: linear-gradient(135deg, #7928ca, #ff0080);
}

.offer:hover {
    transform: translateY(-7px);
}

.offer h3 {
    margin-bottom: 12px;
    font-size: 21px;
}

.offer p {
    line-height: 1.6;
}
```

```
.offer span {
    display: inline-block;
    margin-top: 16px;
    border: 1px dashed white;
    padding: 8px 12px;
    border-radius: 6px;
}

/* Popular routes */
.routes {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 15px;
}

.route {
    padding: 22px;
    border-radius: 12px;
    background: white;
    box-shadow: 0 4px 15px #0000000a;
    transition: .3s;
}

.route:hover {
    box-shadow: 0 8px 22px #d9233222;
    transform: translateY(-4px);
}

.route h3 {
    margin-bottom: 10px;
}

.route p {
    color: #777;
    font-size: 14px;
}

footer {
    background: #242424;
    color: white;
    text-align: center;
    padding: 28px;
    line-height: 1.8;
}

#message {
    margin: 20px auto 0;
    font-weight: bold;
}

@media (max-width: 750px) {
    nav {
        padding: 16px 5%;
    }

    nav a {
        margin-left: 8px;
        font-size: 12px;
    }

    .booking {
        grid-template-columns: 1fr;
    }
```

```
    .offers, .routes {
        grid-template-columns: 1fr;
    }

    .hero {
        padding: 45px 5% 85px;
    }

    .container {
        padding: 35px 5%;
    }
}
</style>
</head>

<body>

<nav>
    <div class="logo">🔴 redBus</div>
    <div>
        <a href="#booking">Book Bus</a>
        <a href="#offers">Offers</a>
        <a href="#routes">Routes</a>
    </div>
</nav>

<section class="hero" id="booking">
    <h1>Your Journey, Our Responsibility 🚌</h1>
    <p>Book your bus tickets easily, quickly, and comfortably.</p>

    <form class="booking" id="busForm">

        <div class="field">
            <label for="from">FROM</label>
            <input id="from" type="text"
                   placeholder="e.g. Pune" required>
        </div>

        <div class="field">
            <label for="to">TO</label>
            <input id="to" type="text"
                   placeholder="e.g. Mumbai" required>
        </div>

        <div class="field">
            <label for="date">JOURNEY DATE</label>
            <input id="date" type="date" required>
        </div>

        <div class="field">
            <label for="passengers">PASSENGERS</label>
            <input id="passengers" type="number"
                   min="1" max="10" value="1" required>
        </div>

        <button class="search-btn" type="submit">
            🔍 Search Buses
        </button>
    </form>

    <div id="message"></div>

    <div class="bus">🚌</div>
    <div class="road"></div>
```

```
</section>

<section class="container" id="offers">
    <h2>🎉 Exclusive Offers</h2>

    <div class="offers">
        <div class="offer">
            <h3>🔥 First Booking</h3>
            <p>Discover special savings on your first bus booking.</p>
            <span>Use code: FIRST</span>
        </div>

        <div class="offer">
            <h3>💰 Weekend Travel</h3>
            <p>Plan your weekend getaway with a great travel deal.</p>
            <span>Use code: WEEKEND</span>
        </div>

        <div class="offer">
            <h3>🎁 Travel More</h3>
            <p>Explore new destinations and plan your next adventure.</p>
            <span>Travel Happy</span>
        </div>
    </div>
</section>

<section class="container" id="routes">
    <h2>📍 Popular Bus Routes</h2>

    <div class="routes">
        <div class="route">
            <h3>🏙️ Pune → Mumbai</h3>
            <p>Explore Maharashtra's popular city connection.</p>
        </div>

        <div class="route">
            <h3>🌆 Mumbai → Goa</h3>
            <p>Plan a relaxing trip to the beaches of Goa.</p>
        </div>

        <div class="route">
            <h3>🏰 Pune → Bengaluru</h3>
            <p>Plan your next journey to India's tech hub.</p>
        </div>

        <div class="route">
            <h3>🌃 Delhi → Jaipur</h3>
            <p>Discover the beautiful Pink City.</p>
        </div>

        <div class="route">
            <h3>🌄 Mumbai → Nashik</h3>
            <p>Explore a popular Maharashtra travel destination.</p>
        </div>

        <div class="route">
            <h3>🌴 Bengaluru → Chennai</h3>
            <p>Travel between two major South Indian cities.</p>
        </div>
    </div>
</section>

<footer>
    <h2>🔴 redBus Travel</h2>
```

```
    <p>Designed for a smooth and comfortable journey.</p>
    <p>Demo website for HTML, CSS and JavaScript practice.</p>
    <p>© 2026 Bus Booking UI Demo</p>
</footer>

<script>
const dateInput = document.getElementById("date");
const today = new Date();
const localToday = new Date(
    today.getTime() - today.getTimezoneOffset() * 60000
).toISOString().split("T")[0];

dateInput.min = localToday;
dateInput.value = localToday;

document.getElementById("busForm").addEventListener("submit",
function(event) {
    event.preventDefault();

    const from = document.getElementById("from").value.trim();
    const to = document.getElementById("to").value.trim();
    const date = dateInput.value;
    const passengers = document.getElementById("passengers").value;
    const message = document.getElementById("message");

    if (from.toLowerCase() === to.toLowerCase()) {
        message.textContent = "Please choose different source and destination cities.";
        return;
    }

    message.textContent =
        "✅ Demo search: " + from + " → " + to +
        " | Date: " + date +
        " | Passengers: " + passengers +
        ". No real buses or tickets are connected.";
});
</script>

</body>
</html>
```

Create the Dockerfile

Create a file named exactly **Dockerfile** with no extension:

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

FROM
nginx:alpine
lightweight Nginx image

COPY
index.html
into the web root

EXPOSE
80
documents the web port

No separate build tool is needed to package this static website.

Push the code

```
$ git init
$ git add index.html Dockerfile
$ git commit -m "First commit"
$ git branch -M main
$ git remote add origin https://github.com/mayurchaudhari0209-collab/<repository>.git
$ git push -u origin main
```

COMMAND	PURPOSE
`git init`	Turns the folder into a Git repository.
`git add index.html Dockerfile`	Stages both files for commit.
`git commit -m "First commit"`	Records a snapshot with a message.
`git branch -M main`	Names the branch `main`.
`git remote add origin <url>`	Links the local repository to the GitHub repository.
`git push -u origin main`	Uploads the code; `-u` remembers the upstream.

Authentication
GitHub no longer accepts account passwords for Git operations. Use a Personal Access Token when prompted, and make sure the remote URL points to `CICD-Project-redbus`.

06 Create a Docker Hub Access Token

An access token is used instead of the account password so access can be limited and revoked independently.

1. In Docker Hub open **Account settings → Personal access tokens**.
2. Click **Generate new token** and enter a description (`Docker-credinals`).
3. Set the expiration (optional) and choose **Repo Read & Write** permission so images can be pushed.
4. Click **Generate** and copy the token to a safe place.

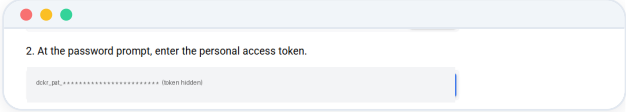


Figure 16. Copy the token (value hidden in this document)

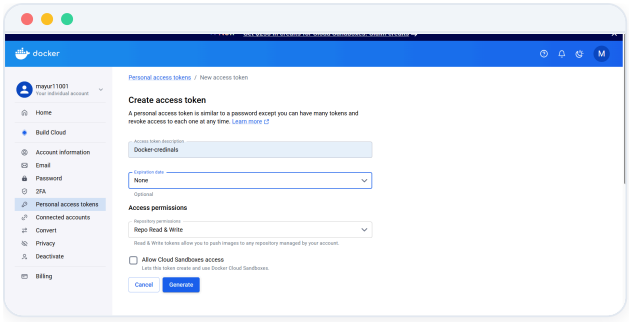


Figure 15. Create access token

> **Keep the token secret**
> Never commit the token to GitHub or place it in a Dockerfile or script. If a token has been shared or appears in a screenshot, revoke it in Docker Hub and generate a new one.

07 Store Credentials in Jenkins

1. Go to **Manage Jenkins → Credentials → System → Global credentials (unrestricted)**.
2. Click **Add Credentials** and choose **Username with password**.
3. Complete the form below and click **Create**.

FIELD	VALUE
Scope	Global (Jenkins, nodes, items, all child items)
Username	Your Docker Hub username
Password	The Docker Hub access token
ID	`docker-creds`
Description	Dockerhub-credinals

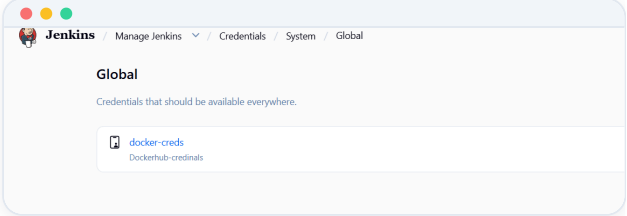


Figure 18. docker-creds listed under Global credentials

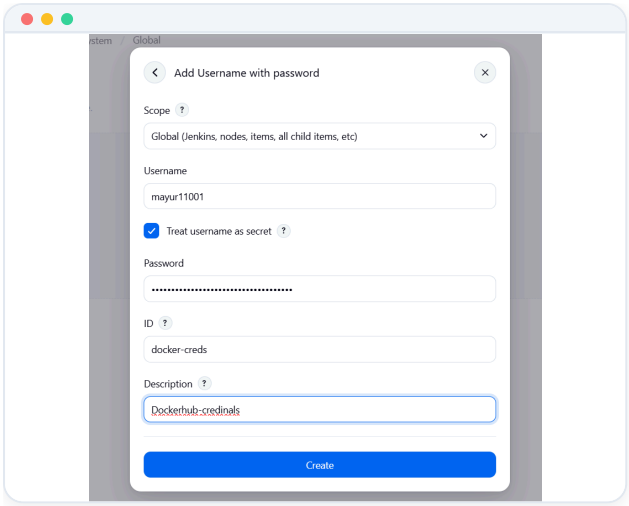


Figure 17. Add Username with password

08 Create and Configure the Freestyle Job

Create the job

From the dashboard select **New Item**, enter a job name, choose **Freestyle project** and click **OK**.

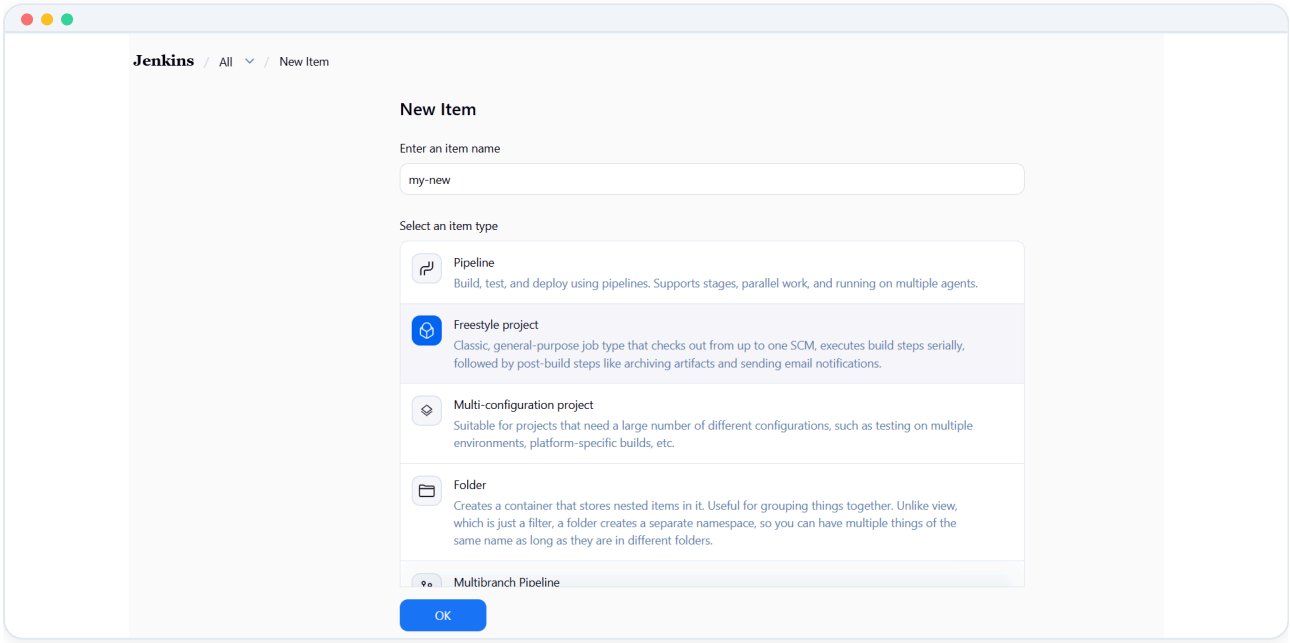


Figure 19. New Item: Freestyle project

Connect the job to GitHub

Under **Source Code Management** select **Git** and enter the repository URL. Because the repository is public, **Credentials** can remain **None**.

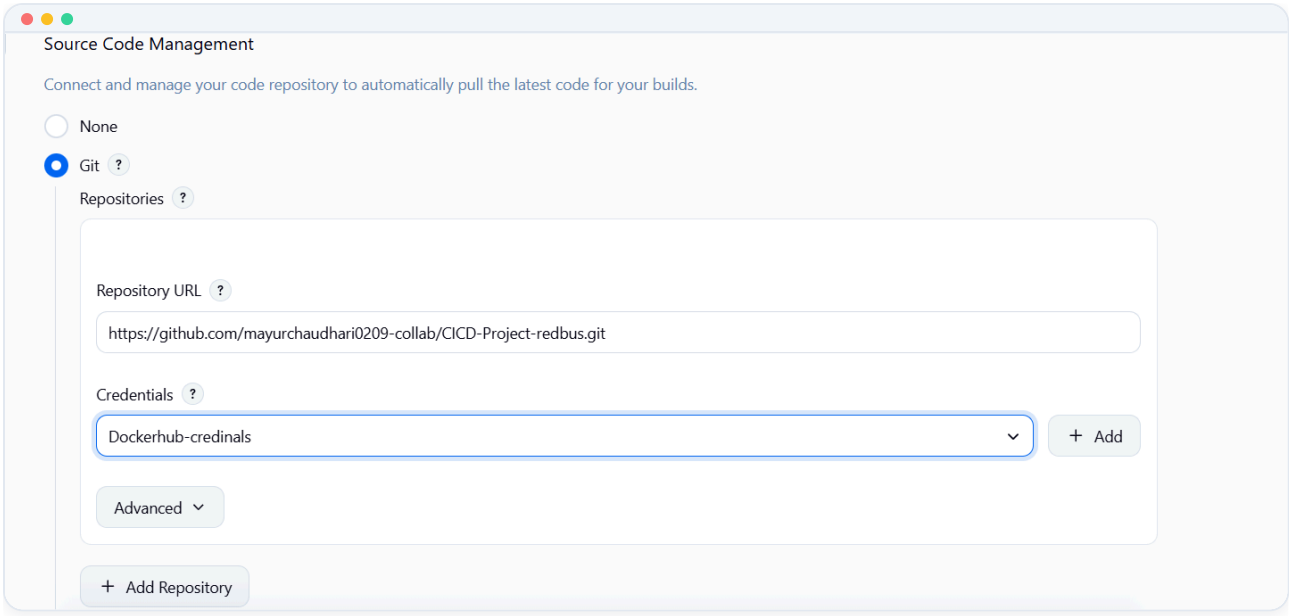


Figure 20. Source Code Management: Git repository URL

Set the **Branch Specifier** to `*/main` to match the branch pushed earlier.

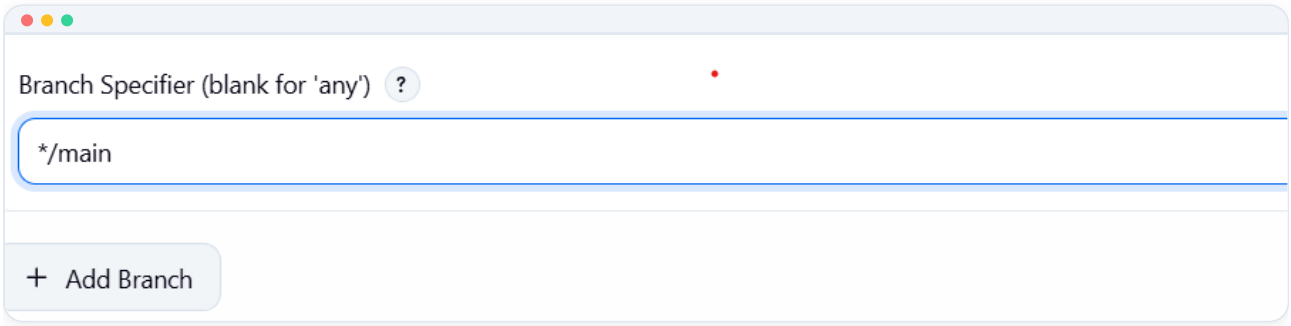


Figure 21. Branch specifier */main

Bind the Docker Hub credentials

Under **Build Environment** enable **Use secret text(s) or file(s)** and add a **Username and password (separated)** binding. Jenkins injects the values as environment variables at build time, so no secret appears in the script.

FIELD	VALUE
Username Variable	`DOCKERHUB_USER`
Password Variable	`DOCKERHUB_TOKEN`
Credentials	docker-creds

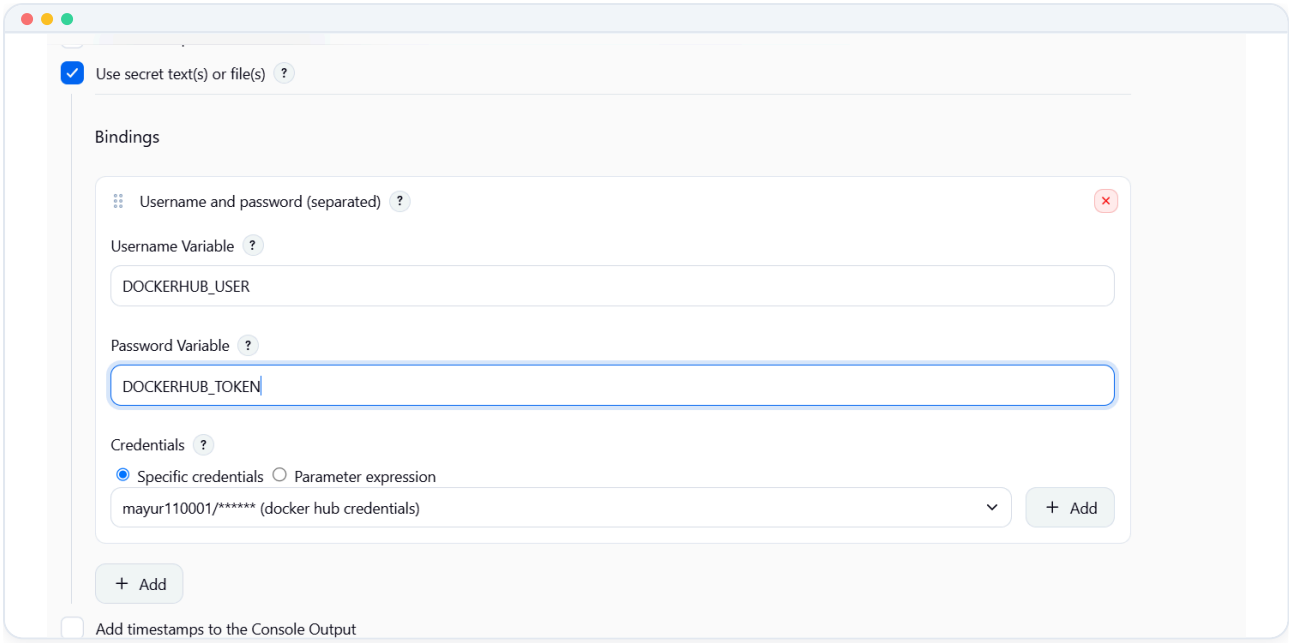


Figure 22. Credentials binding

Add the build step

Under **Build Steps** choose **Add build step → Execute shell** and paste the pipeline script. Click **Apply** and then **Save**.

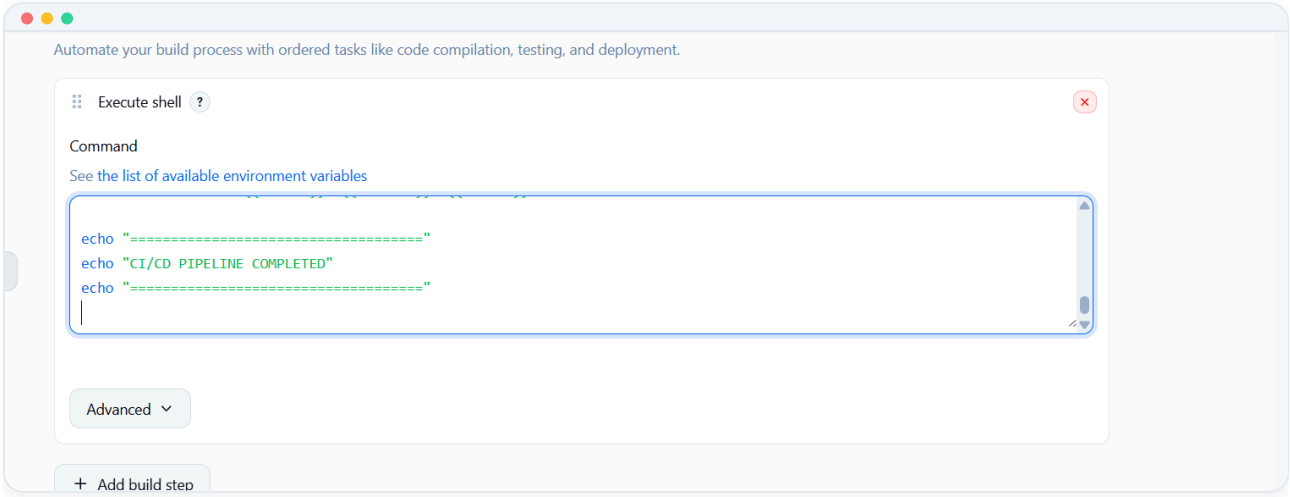


Figure 23. Execute shell build step

Pipeline script

This is the complete script used in the Execute shell step. It pushes to the Docker Hub repository `cloud-messenger-netflix` (create it first, or Docker Hub creates it on the first push to your account) and relies on the `DOCKERHUB_USER` and `DOCKERHUB_TOKEN` variables from the credentials binding.

```
#!/bin/bash
set -Eeuo pipefail

echo "==================================="
echo "STEP 1: Verify source code"
```

```
echo "====================================="

: "${DOCKERHUB_USER:?ERROR: Set DOCKERHUB_USER first}"
: "${DOCKERHUB_TOKEN:?ERROR: Set DOCKERHUB_TOKEN first}"

test -s index.html || {
    echo "ERROR: index.html is missing or empty."
    exit 1
}

test -s Dockerfile || {
    echo "ERROR: Dockerfile is missing or empty."
    exit 1
}

command -v docker >/dev/null 2>&1 || {
    echo "ERROR: Docker is not installed."
    exit 1
}

command -v curl >/dev/null 2>&1 || {
    echo "ERROR: curl is not installed."
    exit 1
}

echo "Website source files verified."

IMAGE="${DOCKERHUB_USER}/cloud-messenger-netflix:latest"

echo "====================================="
echo "STEP 2: Build Docker image"
echo "====================================="

docker build --pull -t "$IMAGE" .

echo "Docker image built successfully."

echo "====================================="
echo "STEP 3: Test Docker image"
echo "====================================="

TEST_CONTAINER="cloudflix-test"
TEST_PORT=18081

cleanup_test() {
    docker rm -f "$TEST_CONTAINER" >/dev/null 2>&1 || true
}

trap cleanup_test EXIT

cleanup_test

docker run -d \
    --name "$TEST_CONTAINER" \
    -p "127.0.0.1:${TEST_PORT}:80" \
    "$IMAGE"

TEST_PASSED=false

for attempt in {1..15}; do
    if curl -fsS "http://127.0.0.1:${TEST_PORT}/" \
        -o /tmp/cloudflix-test.html; then
        if grep -Eqi '<html([[:space:]>])' /tmp/cloudflix-test.html; then
            TEST_PASSED=true
```

```
            break
        fi
    fi
    sleep 2
done

if [[ "$TEST_PASSED" != true ]]; then
    echo "ERROR: Website test failed."
    docker logs "$TEST_CONTAINER" || true
    exit 1
fi

echo "Website test passed."

cleanup_test
trap - EXIT

echo "Test container cleaned up."

echo "===================================="
echo "STEP 4: Login to Docker Hub"
echo "===================================="

if printf '%s' "$DOCKERHUB_TOKEN" | \
    docker login \
        --username "$DOCKERHUB_USER" \
        --password-stdin; then
    echo "Docker Hub login successful."
else
    echo "ERROR: Docker Hub login failed."
    exit 1
fi

echo "===================================="
echo "STEP 5: Push image to Docker Hub"
echo "===================================="

docker push "$IMAGE"

docker logout

echo "Image pushed successfully."

echo "===================================="
echo "STEP 6: Deploy container"
echo "===================================="

docker rm -f cloudflix >/dev/null 2>&1 || true

docker run -d \
    --name cloudflix \
    --restart unless-stopped \
    -p 8081:80 \
    "$IMAGE"

echo "===================================="
echo "STEP 7: Verify deployment"
echo "===================================="

docker ps --filter "name=cloudflix" \
    --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

echo "===================================="
```

```
echo "CI/CD PIPELINE COMPLETED"
echo "=================================="
```

STEP	STAGE	WHAT IT DOES
1	Verify source	Fails fast if the credentials variables are unset, `index.html` or `Dockerfile` is missing or empty, or Docker or curl is not installed.
2	Build image	`docker build --pull` builds `$DOCKERHUB_USER/cloud-messenger-netflix:latest` and refreshes the base image.
3	Test image	Starts a temporary container on `127.0.0.1:18081`, retries up to 15 times (2 s apart) until the page returns HTML, then removes it. The `trap` guarantees cleanup even on failure.
4	Docker Hub login	`docker login --password-stdin` signs in without exposing the token in process lists or logs.
5	Push image	`docker push` uploads the image, then `docker logout` removes the stored login.
6	Deploy	Removes the old `cloudflix` container and starts a new one (`-d` , restart policy `unless-stopped`), mapping host port 8081 to container port 80.
7	Verify deployment	`docker ps` prints the container name, status and ports as a table.

Safe by design
`set -Eeuo pipefail` stops the job on the first error, so a failed test never reaches the push or deploy stages.

09 Run and Verify the Pipeline

Run the build

Open the job and click **Build Now**. A green tick next to build **#1** means every stage succeeded. Open **Console Output** for details; a healthy run ends with `Finished: SUCCESS`.

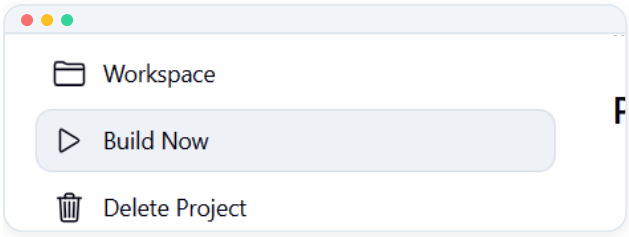


Figure 24. Build Now in the job menu

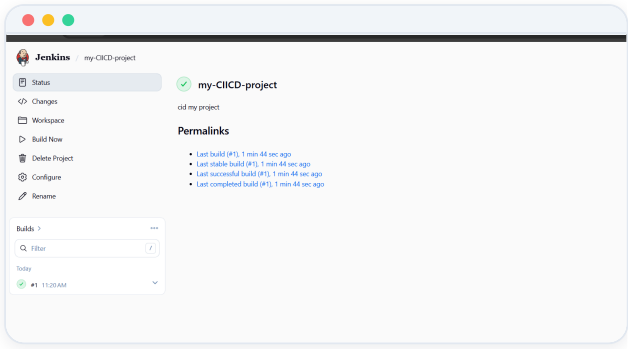


Figure 25. Build #1 completed successfully

Open the website

Browse to `http://<EC2_PUBLIC_IP>:8081`. The redBus landing page confirms that the container built by Jenkins is serving traffic.

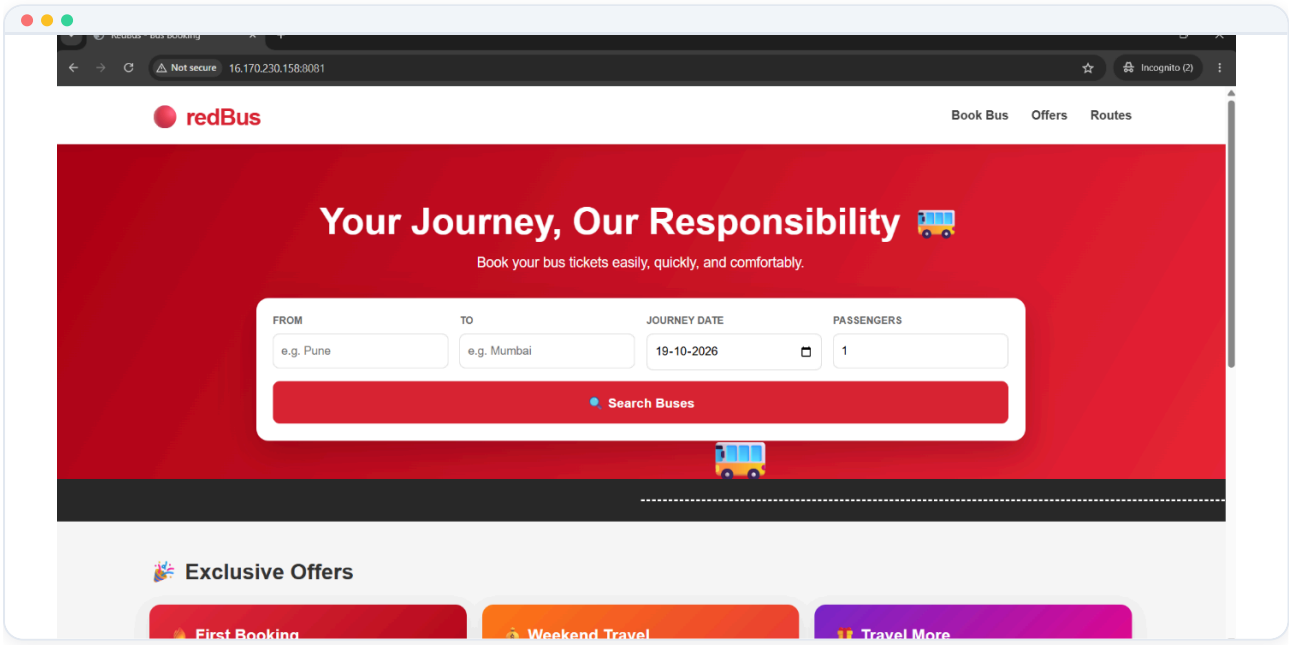


Figure 26. Application live on port 8081

Deployment complete
GitHub source was built by Jenkins, published to Docker Hub and deployed as a running container on AWS EC2.

Test continuous delivery

Edit `index.html` locally, then push the change and click **Build Now** again. Jenkins pulls the new code, rebuilds and pushes the image and redeploys the container.

```
$ git add index.html
$ git commit -m "Update landing page"
$ git push origin main
```

10 Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Jenkins page does not load on :8080	Port 8080 not open or service stopped	Check the security group inbound rule and run `systemctl status jenkins`.
`permission denied` on docker commands in a job	jenkins user not in docker group, or Jenkins not restarted	Run `usermod -aG docker jenkins` then `systemctl restart jenkins`.
Docker login fails in the build	Wrong token or binding variable names	Recreate the token; confirm `DOCKERHUB_USER` and `DOCKERHUB_TOKEN` match the script.
Website does not open on :8081	Port closed or container not running	Check the rule, then `docker ps`, `docker logs cloudflix`, `curl -I http://localhost:8081`.
`git remote add` reports an error	URL pasted without a space after `origin`	Use `git remote add origin <url>` with a space.
`systecmtl: command not found`	Typo	The command is `systemctl`.
Jenkins "Option missing" for secret bindings	Credentials Binding plugin not loaded	Install the plugin from Manage Jenkins → Plugins and restart Jenkins.

11 Security Recommendations

- Restrict ports 22 and 8080 to trusted IP addresses; avoid `0.0.0.0/0`.
- Use access tokens with minimum permissions and rotate them regularly.
- Serve Jenkins over HTTPS and enable role-based access control.
- Treat `docker` group membership as root access; prefer isolated build agents in production.
- Run Jenkins and the application on separate hosts, and deploy to a managed service such as ECS or EKS for production workloads.
- Never place secrets in repositories, Dockerfiles or screenshots.

12 Appendix A: Command Reference

Cleaned command history of the project, in execution order. Typos from the original shell history have been corrected.

#	COMMAND	PURPOSE
01	`sudo su`	Switch to root
02	`yum update -y`	Update all packages
03	`sudo dnf install -y java-21-amazon-corretto`	Install Java 21
04	`sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo`	Add Jenkins repository
05	`sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key`	Import signing key
06	`sudo dnf install -y jenkins`	Install Jenkins
07	`sudo systemctl start jenkins`	Start Jenkins
08	`sudo systemctl enable jenkins`	Start Jenkins at boot
09	`sudo systemctl status jenkins`	Check Jenkins status
10	`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`	Read initial admin password
11	`sudo dnf install -y git`	Install Git
12	`git --version`	Verify Git
13	`yum install docker -y`	Install Docker
14	`docker -v`	Verify Docker
15	`systemctl start docker`	Start Docker
16	`systemctl enable docker`	Start Docker at boot
17	`systemctl status docker`	Check Docker status
18	`sudo usermod -aG docker jenkins`	Add jenkins to docker group
19	`sudo usermod -aG docker ec2-user`	Add ec2-user to docker group
20	`docker ps`	List running containers
21	`sudo systemctl restart jenkins`	Apply group change
22	`mkdir cicd-project`	Create project folder

23	`cd cicd-project`	Enter folder
24	`vi index.html`	Write the website page
25	`vi Dockerfile`	Write the Dockerfile
26	`cat Dockerfile`	Review the Dockerfile
27	`docker build -t jiohostar .`	Build image locally
28	`git init`	Initialise repository
29	`git add index.html Dockerfile`	Stage files
30	`git commit -m "First commit"`	Commit
31	`git branch -M main`	Name branch main
32	`git remote add origin https://github.com/<user>/<repo>.git`	Link GitHub repository
33	`git push -u origin main`	Push to GitHub
34	`history`	Show shell command history

13 Appendix B: Port Reference

PORT	SERVICE	USED FOR
22	SSH	Remote shell access to the instance
80 / 443	HTTP / HTTPS	Created by the default launch wizard rules
8080	Jenkins	Jenkins web interface
8081	Application	redBus website container (maps to port 80 inside the container)

End of document